CSS4005 — Cloud Infrastructure Construction

Lecture 7: AWS Databases: Amazon RDS

Nauruzbayeva Farikha · Lesson 7

Recap — Lesson 6 (AWS IAM and Cloud Security)

- When do you use a role instead of a user? (when something — an EC2 instance, a CI job, a person switching accounts — should borrow temporary credentials instead of owning long-term keys)
- An `Allow` and an explicit `Deny` both match a request. Who wins? (the explicit deny — always)
- How does an app on EC2 get AWS credentials without an access key? (an instance profile: the role's temporary credentials come from the instance metadata service and rotate automatically)
- Security group vs network ACL — which one is stateful? (the security group: return traffic is allowed automatically)

Today’s agenda

- ☐ Database on EC2 vs managed RDS — who does what
- ☐ Engines, instance classes and storage
- ☐ Putting RDS in private subnets, behind security groups
- ☐ Connecting from an app server, and handling credentials
- ☐ Encryption at rest and in transit
- ☐ Backups, point-in-time restore, snapshots
- ☐ Multi-AZ vs read replicas
- ☐ Maintenance, parameter groups, monitoring, cost

The scenario

Your team’s app stores orders in PostgreSQL, installed by hand on the same EC2 instance as the web server. Port 5432 is open to the internet "so the backend developer can connect from home". Backups? Someone ran `pg_dump` once, in September.

On a Friday night the instance’s disk fills up, the database crashes, and the only copy of last month’s orders is on that disk.

Today: let AWS run the database machinery, keep it off the internet, and make sure you can always get your data back.

Block 1

Why a managed database

Self-managed on EC2 vs Amazon RDS

Task	DB on EC2	Amazon RDS
Hardware, host, hypervisor	AWS	AWS
OS + DB engine install & patching	**You**	AWS (in your maintenance window)
Backups & point-in-time restore	**You** (scripts, cron)	AWS (you set retention)
Standby & failover	**You** (replication setup)	AWS (Multi-AZ setting)
Schema, queries, indexes	**You**	**You**
Access, network rules, your data	**You**	**You**

Shared responsibility again: RDS moves the line up, but access control, networking and your data stay on your side.

What you give up

- **No SSH / OS access** to the database host — you can’t install arbitrary extensions or agents. (RDS Custom exists for Oracle and SQL Server when you really need OS access.)
- **No superuser** in the full sense — the master user is powerful, but some engine settings are only reachable through parameter groups.
- Fewer knobs, in exchange for far fewer 3 a.m. pages.

Rule of thumb: if the database is a standard PostgreSQL/MySQL that your app talks to over SQL, RDS is the default. Run it on EC2 only when you need something RDS doesn't allow.

Engines

RDS engines

- PostgreSQL
- MySQL
- MariaDB
- Oracle *(licensing!)*
- Microsoft SQL Server *(licensing!)*
- IBM Db2

Amazon Aurora

- A separate, AWS-built engine family
- **MySQL-compatible** or **PostgreSQL-compatible**
- Cluster storage shared across AZs
- Different pricing and architecture — not just "faster RDS"

In this course we use **RDS for PostgreSQL** (port 5432). Everything also applies to MySQL/MariaDB (port 3306).

Instance classes and storage

- **DB instance class** = CPU + memory, named like EC2: `db.t4g.micro`, `db.m7g.large`, `db.r7g.xlarge`
  - `t` — burstable, for dev/test · `m` — general purpose · `r` — memory-optimized
- **Storage** is EBS under the hood (lecture 5):
  - **gp3** — general-purpose default
  - **io2** — provisioned IOPS for heavy, latency-sensitive databases
- **Storage autoscaling**: set a maximum, and RDS grows storage when free space runs low — no midnight "disk full".

Storage can grow but **cannot shrink** in place. Start small and let autoscaling grow it.

Quick check

Your PostgreSQL on EC2 needs a security patch for the operating system.
Who applies it? And if the same database ran on RDS?

On EC2: you. On RDS: AWS, during the maintenance window you chose. Either way, who can connect to the database is still your job.

Block 2

Network placement

Back to the three-tier VPC (week 4)

```
Internet ──► Web (public subnet)  ──8080──►  App (private subnet)  ──5432──►  DB (DB subnets)
             SG-WEB: 80/443 from 0.0.0.0/0    SG-APP: 8080 from SG-WEB     SG-DB: 5432 from SG-APP
```

- The database lives in **private subnets** — no route to an internet gateway.
- Only the **app tier** may open a connection to it.
- The web tier and the internet **cannot** reach it at all.

That's exactly the policy from the week 4 lab — RDS just plugs into the DB tier.

DB subnet groups

- A **DB subnet group** tells RDS which subnets it may place the database in.
- It must contain subnets in **at least two AZs** — even for a single-AZ database. (That's where a Multi-AZ standby or a failover replacement can go.)
- Use **private** subnets only.

```
aws rds create-db-subnet-group \
  --db-subnet-group-name course-db-subnets \
  --db-subnet-group-description "DB tier, private" \
  --subnet-ids subnet-db-a subnet-db-b     # e.g. 10.0.3.0/24 (AZ a), 10.0.4.0/24 (AZ b)
```

Security group for the database

✅ SG-DB inbound

Type	Port	Source
PostgreSQL	5432	**sg-app**

The source is the **app tier's security group**, not an IP range. Any instance in SG-APP can connect; nothing else can.

❌ Never

Type	Port	Source
PostgreSQL	5432	0.0.0.0/0

...and set **Public access: No** when creating the instance.

"Publicly accessible + 0.0.0.0/0" is how databases end up in breach reports. Developers who need access go through the app tier, a bastion host, or SSM port forwarding — not through an open port.

Creating the instance (CLI)

```
aws rds create-db-instance \
  --db-instance-identifier course-db \
  --engine postgres \
  --db-instance-class db.t4g.micro \
  --allocated-storage 20 --storage-type gp3 \
  --master-username dbadmin --manage-master-user-password \
  --db-subnet-group-name course-db-subnets \
  --vpc-security-group-ids sg-db \
  --no-publicly-accessible \
  --storage-encrypted --backup-retention-period 7
```

Every flag here is a decision from today: engine and size, a password you never see (Secrets Manager), private placement, encryption, backups.

Block 3

Connecting and credentials

The endpoint

- Every DB instance gets a **DNS endpoint**, e.g. `course-db.abc123xyz.eu-central-1.rds.amazonaws.com`, plus a port.
- Your app connects to the **name**, never an IP — the IP can change (failover, maintenance, restore).
- From the app server (inside the VPC):

```
aws rds describe-db-instances --db-instance-identifier course-db \
  --query 'DBInstances[0].Endpoint.[Address,Port]' --output text

psql "host=course-db.abc123xyz.eu-central-1.rds.amazonaws.com port=5432 \
      user=dbadmin dbname=postgres sslmode=require"
```

From your laptop this times out — and that's correct: the DB is private.

Where does the password live?

- **Bad:** hard-coded in `config.py`, committed to Git.
- **OK:** an environment variable injected at deploy time.
- **Good: AWS Secrets Manager**. With `--manage-master-user-password`, RDS generates the master password, stores it in Secrets Manager and can rotate it — you never type it.
- The app reads it at startup, using its **instance profile** (lecture 6) for permission.

```
SECRET=$(aws rds describe-db-instances --db-instance-identifier course-db \
  --query 'DBInstances[0].MasterUserSecret.SecretArn' --output text)
aws secretsmanager get-secret-value --secret-id "$SECRET" \
  --query SecretString --output text
```

IAM database authentication

- For MySQL, MariaDB and PostgreSQL: log in with a short-lived **auth token** instead of a password.
- The token is generated from IAM credentials and is valid for **15 minutes**; the connection must use SSL/TLS.
- Permission is granted with the `rds-db:connect` action in an IAM policy — so database access follows your IAM roles.

```
TOKEN=$(aws rds generate-db-auth-token --hostname <endpoint> \
  --port 5432 --region eu-central-1 --username app_user)
PGPASSWORD="$TOKEN" psql "host=<endpoint> user=app_user dbname=shop sslmode=require"
```

Still create separate DB users per app with only the grants they need — least privilege applies inside the database too.

Quick check

The app server can't connect to RDS: the connection hangs and times out. Name two things to check first.

SG-DB allows 5432 from the app server's security group; the app is in the same VPC (or routed to it) and uses the endpoint name and the right port. A timeout is almost always network, not password.

Block 4

Encryption

Encryption at rest

- Turn on with `--storage-encrypted`, using an AWS-managed or **customer-managed KMS key**.
- Covers the storage, automated backups, snapshots and read replicas.
- Must be chosen **at creation** — you can't flip it on for an existing unencrypted instance.

Encrypting an existing database:

1. Take a snapshot of the unencrypted instance
2. **Copy** the snapshot with encryption and a KMS key
3. Restore a new instance from the encrypted copy, then switch the app over

Encryption in transit

- RDS endpoints support **TLS**. Use `sslmode=require` (PostgreSQL) or `--ssl-mode=REQUIRED` (MySQL client) at minimum.
- To also verify the server's identity, download the RDS CA bundle and use `sslmode=verify-full`: `https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem`
- To **force** TLS on the server side, set a parameter in a parameter group: `rds.force_ssl` (PostgreSQL) or `require_secure_transport` (MySQL).

Inside a private VPC traffic is already isolated — TLS still matters, because "the network is private" is an assumption, not a guarantee.

Block 5

Backups and restore

Automated backups and point-in-time restore

- RDS takes a daily snapshot **and** keeps transaction logs.
- **Retention period**: 1–35 days (0 turns automated backups off — don't).
- **Point-in-time restore (PITR)**: restore to any second inside the retention window, up to the latest restorable time (usually a few minutes ago).
- Automated backups are deleted when retention expires — or when you delete the instance, unless you choose to retain them.

```
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier course-db \
  --target-db-instance-identifier course-db-before-bug \
  --restore-time 2026-10-12T09:00:00Z
```

Manual snapshots

- `create-db-snapshot` — taken when you ask, kept **until you delete it** (independent of the retention period).
- Use before risky changes: an engine upgrade, a big migration, deleting the instance.
- Can be **copied** to another region (disaster recovery) or **shared** with another account.

```
aws rds create-db-snapshot --db-instance-identifier course-db \
  --db-snapshot-identifier course-db-before-migration
```

Restore = a new database

- Restoring from a snapshot or a point in time creates a **new DB instance** with a **new endpoint**.
- It does **not** overwrite the original.
- After restore you must: attach the right security group and parameter group, check settings, then point the app at the new endpoint (or rename instances).

A backup you've never restored is a hope, not a backup. Practise a restore before you need one.

Quick check

At 10:15 someone ran `DELETE FROM orders;` without a `WHERE`. Backups are on with 7-day retention. What do you do, and what changes for the app?

Point-in-time restore to 10:14 — this creates a new instance with a new endpoint. Copy the rows back, or switch the app to the restored instance.

Block 6

Availability and scaling

Multi-AZ: survive an AZ failure

- RDS keeps a **standby** in another AZ, updated by **synchronous** replication — every committed write is on both.
- If the primary fails (host, storage, AZ outage) or during some maintenance, RDS **fails over** automatically.
- The **endpoint stays the same** — DNS now points at the old standby. Apps reconnect; nothing to reconfigure.
- The standby **does not serve reads**. Multi-AZ is for availability, not performance.

(RDS also has a "Multi-AZ DB cluster" option with two readable standbys — a different deployment type. Today we mean the classic Multi-AZ instance.)

Read replicas: scale reads

- A copy kept up to date by **asynchronous** replication — it can lag slightly behind the primary.
- Has its **own endpoint**; the app sends read-only queries (reports, dashboards) there.
- Can live in **another region** — closer to users, or for disaster recovery.
- Can be **promoted** to a standalone database — but that's a manual step, not automatic failover.

Multi-AZ vs read replica

	Multi-AZ (instance)	Read replica
Purpose	**Availability**	**Read scaling**
Replication	Synchronous	Asynchronous
Serves traffic?	No — standby only	Yes — reads
Endpoint	Same as primary	Its own
On primary failure	Automatic failover	Manual promotion
Location	Another AZ, same region	Same or another region

They combine: a production database is often Multi-AZ *and* has one or more read replicas.

Quick check

Monthly reports make the database slow for customers. Your teammate says "turn on Multi-AZ". Is that the fix?

No. The Multi-AZ standby doesn't serve reads. Add a read replica and point the reporting queries at its endpoint.

Block 7

Operations and cost

Maintenance windows and upgrades

- You choose a weekly **maintenance window**; AWS applies OS and engine patches there.
- **Minor version upgrades** can be automatic (opt-in setting).
- **Major version upgrades** (e.g. PostgreSQL 16 → 17) are always your decision: take a snapshot, test on a restored copy, then upgrade.
- With Multi-AZ, many maintenance actions happen on the standby first, shortening the interruption.

Parameter groups

- Engine settings (`max_connections`, `rds.force_ssl`, `log_min_duration_statement`, …) live in a **DB parameter group**, not in a config file you edit.
- The **default** parameter group can't be changed — create your own and attach it.
- **Dynamic** parameters apply right away; **static** ones need a reboot.

```
aws rds create-db-parameter-group --db-parameter-group-name course-pg17 \
  --db-parameter-group-family postgres17 --description "course settings"
aws rds modify-db-parameter-group --db-parameter-group-name course-pg17 \
  --parameters "ParameterName=rds.force_ssl,ParameterValue=1,ApplyMethod=immediate"
```

Monitoring

Tool	Shows
CloudWatch metrics	`CPUUtilization`, `FreeStorageSpace`, `FreeableMemory`, `DatabaseConnections`, `ReadIOPS` / `WriteIOPS`
CloudWatch alarms	Notify you *before* storage or CPU runs out
Enhanced Monitoring	OS-level metrics (processes, memory, disk) from an agent on the host
Performance Insights / CloudWatch Database Insights	Which **queries** are making the database busy
Logs	Engine logs (errors, slow queries) exported to CloudWatch Logs

The first alarm to create on any database: `FreeStorageSpace` below a threshold. Disk-full was the scenario at the start of today.

Cost hygiene

- You pay for **instance hours**, **storage**, **backup storage** beyond the free allowance, Multi-AZ (roughly a second instance) and data transfer.
- A **stopped** RDS instance still pays for storage — and RDS **starts it again automatically after 7 days**.
- Delete what you don't need. On delete you choose: **final snapshot** (kept, billed) or none, and whether to retain automated backups.
- Turn on **deletion protection** for anything real.
- Free tier terms changed for accounts created after July 15, 2025 — check `aws.amazon.com/free` before you launch.

Try it yourself

With AWS

1. DB subnet group with two private subnets
2. SG-DB: 5432 from your app server's SG only
3. `db.t4g.micro` PostgreSQL, gp3, encrypted, not public, managed password
4. `psql` from the EC2 app server, create a table
5. Manual snapshot → restore → connect to the new endpoint
6. **Delete everything**, including snapshots

Without AWS

1. PostgreSQL in a Linux VM (`apt install postgresql`) or a container (`postgres:17`)
2. Create a DB user with only the grants an app needs
3. Allow connections from one IP only (`pg_hba.conf`)
4. `pg_dump` = your "snapshot"; drop the table, then restore it with `psql`
5. Compare: what would RDS have done for you?

By the end of this lesson, you should be able to

- ☐ Explain what RDS manages for you, and what stays your job
- ☐ Place an RDS instance in private subnets with a DB subnet group and an SG that only trusts the app tier
- ☐ Connect from an app server using the endpoint, without hard-coded passwords
- ☐ Choose encryption at rest and in transit, and know when it must be decided
- ☐ Restore a database to a point in time — and know that you get a new endpoint
- ☐ Pick Multi-AZ or a read replica for a given problem

Before next lecture

- ☐ If you created RDS resources: delete instances **and** manual snapshots
- ☐ Review weeks 1–7: VPC, EC2, EBS/S3, IAM, RDS — the midterm covers all of them
- ☐ Sketch the three-tier architecture from memory: subnets, route tables, security groups, where RDS sits

Next week puts every piece so far into one architecture — and that's what the midterm asks you to design and explain.

Next lecture

Midterm and AWS Architecture — putting VPC, EC2, storage, IAM and RDS together into one design you can defend.